

Student OS — Final Specification (Hackathon Edition)

Version: 1.0

Status: FINAL & LOCKED

Purpose: Single Source of Truth for all future prompts, architecture decisions, implementation, testing, and Kaggle submission.

A. LOCKED SCOPE

Project Name

Student OS — Academic Co-Pilot

Competition

Kaggle AI Agents: Intensive Vibe Coding Capstone Project

Project Vision

Student OS is an AI-powered academic co-pilot that maintains a persistent student profile, proactively detects academic risk, identifies scholarship opportunities, and generates weekly academic briefings.

The system demonstrates agentic behavior by acting on student data without requiring explicit user requests.

Core Objective

Build a practical AI Agent system that demonstrates:

- Multi-Agent Architecture
- Persistent Memory
- Session Management
- Agent Skills
- Proactive Reasoning
- Explainable Decisions
- Security Controls

while remaining simple enough for a solo developer to build within 14 days.

Locked Agent Count

Exactly 3 agents.

Agent 1

Orchestrator Agent

Agent 2

Academic Risk Agent

Agent 3

Opportunity Scout Agent

No additional agents may be added.

Locked Skill Count

Exactly 3 skills.

Skill 1

GPA Risk Assessment

Skill 2

Scholarship Matching

Skill 3

Weekly Briefing Generation

No additional skills may be added.

Locked Memory Files

Exactly 3 persistent memory files.

```
student_profile.json  
activity_log.json  
alert_history.json
```

Technology Stack

Frontend:

- Streamlit

LLM:

- Gemini 2.5 Flash

Agent Framework:

- Google ADK

Persistence:

- JSON Files

Language:

- Python
-

Out of Scope

The following are intentionally excluded:

- Career Agent
 - Resume Builder
 - LMS Integration
 - Canvas Integration
 - Blackboard Integration
 - Mobile App
 - Professor Matching
 - Peer Benchmarking
 - Learning Style Analysis
 - Knowledge Gap Analysis
 - Stress Prediction
 - Calendar Synchronization
 - OAuth
 - Database Systems
 - Vector Databases
 - RAG Pipelines
 - Multi-Tenant Architecture
-

B. FINAL MVP

One-Sentence Description

A three-agent academic co-pilot that maintains a persistent student profile, proactively detects academic risk, and surfaces scholarship opportunities without requiring the student to ask.

Primary User

University Student

Core Features

Student Profile

Persistent student profile containing:

- Name
 - Major
 - GPA
 - Target GPA
 - Credits Completed
 - Graduation Goal
 - Career Goal
 - Study Hours Per Week
 - Scholarship Preferences
-

Proactive Risk Alert

Automatically generated when:

- GPA falls below target
- GPA reaches risk threshold
- Alert threshold exceeded

No user prompt required.

Scholarship Matching

Automatically scans scholarship dataset.

Returns:

- Top matches

- Match scores
 - Eligibility reasoning
-

Weekly Briefing

Generated automatically.

Includes:

- Risk Summary
 - Opportunity Summary
 - Top Priorities
-

Agent Activity Log

Displays:

- Agent Name
- Action
- Reason
- Result
- Timestamp

Provides visible proof of agent coordination.

Session Persistence

Student profile survives:

- Browser refresh
 - Session restart
-

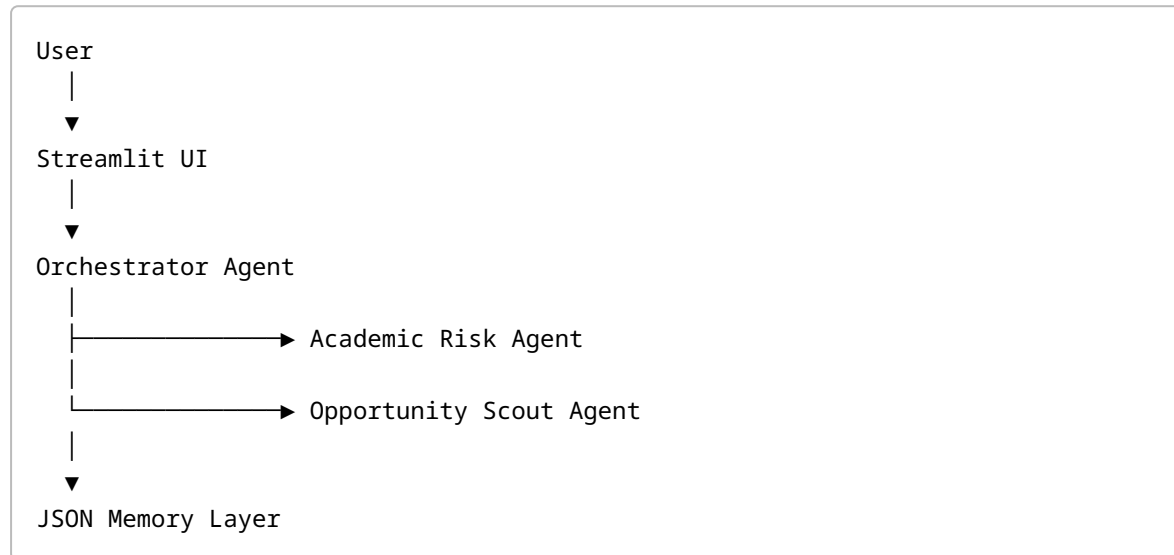
Success Criteria

The demo must show:

1. Memory persistence
 2. Agent coordination
 3. Skill reuse
 4. Proactive alerts
 5. Scholarship recommendations
 6. Security controls
-

C. ARCHITECTURE

High-Level Architecture



Agent Responsibilities

Orchestrator Agent

Responsibilities:

- User communication
- Agent routing
- Context assembly
- Memory loading
- Memory updates
- Weekly briefing generation
- Security enforcement

Permissions:

READ:

- student_profile.json
- activity_log.json
- alert_history.json

WRITE:

- student_profile.json
- activity_log.json
- alert_history.json

Academic Risk Agent

Responsibilities:

- GPA analysis
- Risk scoring
- Risk recommendations
- Alert generation

Permissions:

READ ONLY

No write access.

Opportunity Scout Agent

Responsibilities:

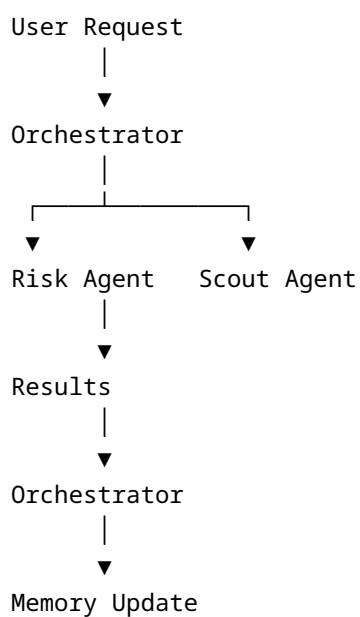
- Scholarship discovery
- Scholarship ranking
- Match explanation

Permissions:

READ ONLY

No write access.

Coordination Flow



↓
UI Response

D. MEMORY

Memory Philosophy

One long-term brain.

One activity history.

One alert history.

One temporary session buffer.

Memory Types

Session Memory

Stored in:

```
st.session_state
```

Contains:

- Current interaction
- Temporary reasoning
- Agent intermediate outputs

Not persisted.

Long-Term Memory

Stored in:

```
student_profile.json
```

Contains:

- Identity
- GPA
- Goals

- Preferences
 - Academic baseline
-

Activity Memory

Stored in:

activity_log.json

Contains:

- Queries
 - Agent actions
 - Recommendations
 - Searches
 - Briefings
-

Alert Memory

Stored in:

alert_history.json

Contains:

- Risk alerts
 - Notifications
 - Academic warnings
-

Memory Ownership

READ:

- All agents

WRITE:

- Orchestrator only
-

Memory Retrieval Flow

Load Profile
Load Activity
Load Alerts
Load Session Context

↓

Build Context

↓

Route Agent

↓

Generate Response

Memory Update Flow

Agent Output

|

▼

Orchestrator Validation

|

├─ Update Activity Log

|

├─ Update Profile

|

└─ Update Alerts

E. SKILLS

Skill Philosophy

Skills contain business logic.

Agents invoke skills.

Skills are reusable.

Skill 1

GPA Risk Assessment Skill

Purpose:

Evaluate academic risk.

Inputs:

- GPA
- Target GPA
- Credits Completed
- Alert History

Outputs:

- Risk Score
- Risk Level
- Recommendations

Rules:

GPA < 2.5 → HIGH

GPA < Target GPA - 0.5 → MEDIUM

Otherwise → LOW

Used By:

- Academic Risk Agent
- Orchestrator Agent

Skill 2

Scholarship Matching Skill

Purpose:

Match scholarships to student profile.

Inputs:

- GPA
- Major

- Year
- Preferences

Outputs:

- Match Score
- Eligibility Reason
- Ranked Scholarships

Used By:

- Opportunity Scout Agent
 - Orchestrator Agent
-

Skill 3

Weekly Briefing Generation Skill

Purpose:

Generate proactive academic summary.

Inputs:

- Student Profile
- Activity History
- Alert History
- Scholarship Matches

Outputs:

- Top Priorities
- Risk Summary
- Opportunity Summary

Priority Order:

1. Risk Alerts
2. Scholarship Deadlines
3. Academic Recommendations

Used By:

- Orchestrator Agent
-

Skill Reuse Matrix

Skill	Risk Agent	Scout Agent	Orchestrator
GPA Risk Assessment	✓		✓
Scholarship Matching		✓	✓
Weekly Briefing			✓

F. SECURITY

Security Philosophy

Demonstrate security awareness without enterprise complexity.

Threat Model

Threat 1

Malicious User Input

Threat 2

Prompt Injection

Threat 3

Memory Corruption

Threat 4

Agent Privilege Escalation

Threat 5

Session Leakage

Security Controls

Control 1

Input Validation

Checks:

- Empty Input
 - Maximum Length
 - Allowed Characters
 - Schema Validation
-

Control 2

Orchestrator Write Gate

Only Orchestrator may write memory.

Agents cannot directly modify files.

Control 3

Session Isolation

Uses:

```
st.session_state
```

Session data is not shared.

Control 4

Prompt Injection Filter

Block patterns:

```
ignore previous instructions  
reveal system prompt  
show hidden memory  
delete memory  
override rules  
act as administrator  
print your instructions
```

Control 5

Structured Outputs

All agents return structured JSON.

Example:

```
{
  "agent": "AcademicRiskAgent",
  "status": "success",
  "result": {},
  "proposed_update": null
}
```

Schema Locking

Allowed Profile Fields:

```
student_id
name
major
gpa
target_gpa
credits_completed
target_graduation
career_goal
study_hours_per_week
preferences
risk_baseline
```

Unknown fields are rejected.

G. IMPLEMENTATION PLAN

Phase 1

Foundation

Days 1-3

Deliverables:

- Streamlit Skeleton
 - JSON Memory
 - Profile Forms
 - Session State
-

Phase 2

Agent Infrastructure

Days 4-6

Deliverables:

- ADK Setup
 - Orchestrator Agent
 - Agent Routing
 - Context Builder
-

Phase 3

Skills Implementation

Days 7-9

Deliverables:

- GPA Risk Skill
 - Scholarship Matching Skill
 - Weekly Briefing Skill
-

Phase 4

Visibility Layer

Days 10-11

Deliverables:

- Agent Activity Log
 - Reasoning Display
 - Status Indicators
-

Phase 5

Security

Days 12-13

Deliverables:

- Input Validation
 - Prompt Guard
 - Schema Validation
 - Write Gate Enforcement
-

Phase 6

Finalization

Day 14

Deliverables:

- Testing
 - Demo Recording
 - README
 - Kaggle Submission
-

Final Repository Structure Target

```
student-os/  
  
app.py  
  
agents/  
|  
├── orchestrator.py  
├── academic_risk_agent.py  
└── opportunity_scout_agent.py  
  
skills/  
|  
├── gpa_risk/  
│   ├── skill.md  
│   └── skill.py  
├── scholarship_matching/  
│   ├── skill.md  
│   └── skill.py  
└── weekly_briefing/  
    ├── skill.md  
    └── skill.py  
  
memory/
```

```
|
|— student_profile.json
|— activity_log.json
|— alert_history.json

security/
|
|— validators.py
|— prompt_guard.py
|— security_rules.py

utils/
|
|— memory_ops.py
|— logger.py
```

Final Judge Criteria Mapping

Criterion	Demonstration
Multi-Agent	Orchestrator + Risk + Scout
Memory	JSON Persistence
Sessions	Streamlit Session State
Skills	3 Reusable Skills
Security	5 Security Controls
Proactive Behavior	Risk Alerts + Weekly Briefing
Explainability	Agent Activity Log
Persistence	Profile Survives Refresh

FINAL LOCK

This specification is considered FINAL.

Future prompts must NOT:

- Add agents
- Add skills
- Add databases
- Add vector stores
- Add integrations
- Change architecture

Future prompts may ONLY:

- Refine implementation
- Generate code
- Generate datasets
- Generate UI
- Generate tests

without changing this specification.